EBOOK — Controle Financeiro do Zero

Tutorial guiado, microetapa por microetapa.

> Projeto didático: um site responsivo de controle financeiro pessoal (receitas, despesas e saldo), construído do zero, ensinando programação para iniciantes absolutos.

Como funciona este tutorial

- Avançamos **uma microetapa por vez**: objetivo → implementação → explicação → teste → resultado.
- Linguagem acessível: cada conceito novo é explicado primeiro em português simples.
- Usa apenas HTML, CSS e JavaScript puro — sem instalar nada além de um editor (VSCode, opcional).

Sumário

1. Fase 1 — Estrutura e interface básica (4 microetapas)
2. Fase 2 — Cadastro de transações (6 microetapas)
3. Bugs corrigidos (saldo após F5 e transação sem valor)
4. Fase 3 — Categorias e organização (3 microetapas)
5. Fase 4 — Resumo e total por categoria (3 microetapas)
6. Fase 5 — Aperfeiçoamentos e diferenciais (2 microetapas)
7. Fase 6 — Entrada do site: landing + cadastro/login (6 microetapas)
8. GitHub: subindo o projeto para a nuvem
9. Glossário: conceitos aprendidos
10. Próximos passos

FASE 1 — ESTRUTURA E INTERFACE BÁSICA

Microetapa 1 — A estrutura de arquivos

Objetivo: criar a "casa" do projeto: uma pasta com 4 arquivos.

Arquivo	Função
index.html	Estrutura da página (textos, campos, botões)
style.css	Visual (cores, espaçamento, responsividade)
app.js	Lógica (somar, guardar, mostrar dados)
PROGRESSO.md	Memória do projeto (diário de desenvolvimento)

Conceito novo — extensão de arquivo: é o final do nome (`html`, `css`, `js`). Diz ao computador o tipo do arquivo e qual programa sabe abrir. O `index.html` é sempre a porta de entrada de um site.

Microetapa 2 — A base do HTML

Objetivo: montar o esqueleto: cabeçalho e área principal.

```
<body>
  <header>
    <h1>Meu Controle Financeiro</h1>
  </header>
  <main>
    <p>Aqui vão ficar seus registros de receitas e despesas.</p>
  </main>
</body>
```

- `<header>` → topo da página (cabeçalho). Tag semântica.
- `<h1>` → título grande.
- `<main>` → área principal do conteúdo.
- `<p>` → parágrafo.

Conceito novo — tags em pares: `<header>` abre e `</header>` fecha. Como caixas: o que está dentro pertence à caixa, e caixas podem conter outras (o `<h1>` dentro do `<header>`).

Microetapa 3 — O primeiro CSS

Objetivo: dar cores e organização; funcionar em PC e celular.

Conexão ao HTML: `<link rel="stylesheet" href="style.css">`

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  background-color: #f4f4f9;
  color: #333;
}

header {
  background-color: #4a90e2;
  color: white;
  padding: 20px;
  text-align: center;
```

```
}

@media (min-width: 768px) {
  main {
    max-width: 700px;
    margin: 0 auto;
  }
}
```

- body → regras gerais. `margin: 0` remove espaços automáticos; `#f4f4f9` é **código de cor hexadecimal** (começa com #).
- `padding` → espaço interno da caixa.
- `text-align: center` → centraliza texto.

Conceito novo — media query (responsividade): `@media (min-width: 768px)` vale somente quando a tela tem pelo menos 768px. No celular o conteúdo ocupa a largura toda; no PC fica estreito e centralizado (`margin: 0 auto`).

Microetapa 4 — O primeiro JavaScript

Objetivo: JS escreve o saldo na tela. Vale para o projeto inteiro: **HTML define o lugar, CSS enfeita, JS comanda.**

```
<p>R$ <span id="saldo"></span></p>
<script src="app.js"></script>
```

```
var saldo = 0;

function exibirSaldo() {
  document.getElementById("saldo").textContent = "R$ " + saldo.toFixed(2).replace(".",
}

exibirSaldo();
```

- `<span id="saldo">` → caixinha com um **endereço** (`id`) que o JS acha.
- `<script>` → carrega o JS (como o `<link>` carrega o CSS).
- `var saldo = 0;` → **variável**: caixa na memória que guarda um valor.

- `function exibirSaldo() {...}` → **função**: bloco que executa quando chamado.
- `document.getElementById("saldo")` → o JS pega o elemento pelo id.
- `.textContent` → substitui o texto dentro do elemento.
- `toFixed(2)` → 2 casas decimais. `.replace(".", ",")` → formato brasileiro.
- `exibirSaldo();` → **chamada da função** — sem ela, a função não roda.

FASE 2 — CADASTRO DE TRANSAÇÕES

Microetapa 1 — O formulário de receita

Objetivo: criar a tela para digitar descrição e valor de uma receita.

```
<form id="form-receita">
  <label for="descricao">Descrição</label>
  <input type="text" id="descricao" placeholder="Ex: Salário">
  <label for="valor">Valor (R$)</label>
  <input type="number" id="valor" step="0.01" min="0">
  <button type="submit">Adicionar</button>
</form>
```

- `<form>` → formulário; ganhou `id` para o JS achar.
- `<label for="...">` → etiqueta ligada ao campo de mesmo `id`.
- `input type="text"` → campo de texto; `placeholder` = sugestão que some ao digitar.
- `input type="number"` → só números; `step="0.01"` = centavos; `min="0"` barra negativos.
- `button type="submit"` → botão que envia; o JS vai interceptar.

Microetapa 1b — O visual dark + laranja

Objetivo: tema escuro com laranja, inspirado no Meu Planner Financeiro.

Paleta: fundo `#111827`, cartões `#1f2937`, destaque `#f97316`, texto `#f9fafb`.

- **Dark mode cria profundidade:** o que está por cima é mais claro que o fundo.
- **Seletor combinado** `header h1` = o `<h1>` dentro do `<header>`.
- `border-radius` nos `<section>` → cantos arredondados viram cartões.
- Contraste: campo escuro com texto claro (legibilidade).
- `#saldo` com `font-size: 2em` e `font-weight: bold` → saldo em destaque.

Microetapa 2 — O botão Adicionar funciona

Objetivo: o clique pega o valor digitado, soma ao saldo e mostra na tela.

```
document.getElementById("form-receita").addEventListener("submit", function (evento) {
  evento.preventDefault();

  var descricao = document.getElementById("descricao").value;
  var valor = Number(document.getElementById("valor").value);

  saldo = saldo + valor;
  exibirSaldo();
});
```

- `addEventListener("submit", ...)` → o JS **escuta** o evento de envio.
- `evento.preventDefault()` → **impede recarregar a página** (comportamento padrão).
- `.value` → lê o que foi digitado.
- `Number(...)` → converte texto em número: `"250" + 100` não soma, vira `"250100"`.
- `var descricao` → guardada para a próxima etapa (lista de transações).

Conceito novo — evento: um sinal do navegador ("alguém clicou"); o JS escuta e reage.

Microetapa 3 — O formulário de despesa

Objetivo: cadastrar despesas — mesma estrutura da receita, mas **subtrai** valor.

Os campos usam **ids diferentes** (`descricao-despesa`, `valor-despesa`) porque **ids devem ser únicos** na página.

```
document.getElementById("form-despesa").addEventListener("submit", function (evento) {
  evento.preventDefault();
  var valor = Number(document.getElementById("valor-despesa").value);
  saldo = saldo - valor;
  exibirSaldo();
});
```

Lição sobre padrão: copiamos uma estrutura parecida e ajustamos o essencial (o `id` e o sinal `-`). Mais adiante vamos reduzir a repetição.

Microetapa 3b — Salvando com localStorage

Objetivo: os dados sobrevivem ao F5, sem servidor.

```
var saldo = Number(localStorage.getItem("saldo")) || 0;

function exibirSaldo() {
  document.getElementById("saldo").textContent = "R$ " + saldo.toFixed(2).replace(".",
  localStorage.setItem("saldo", saldo);
}
```

- `localStorage.getItem("saldo")` → lê da "gaveta" do navegador (texto ou `null`).
- `Number(...)` → converte de volta para número (a gaveta só guarda texto).
- `|| 0` → "ou então 0": sem nada guardado, começa em 0.
- `localStorage.setItem("saldo", saldo)` → grava; ficou dentro de `exibirSaldo()`, que roda a cada mudança → sempre que muda, salva (um só lugar para salvar evita bugs).

Conceito novo — persistência: guardar informações para além do momento em que o programa roda.

BUG CORRIGIDO — saldo não aparecia após F5

Sintoma: após atualizar, o saldo sumia; ao fazer uma transação nova, somava com o valor antigo.

Diagnóstico: a gaveta (localStorage) sempre funcionou. Faltava **pintar o número na tela ao abrir a página**: a chamada inicial `exibirSaldo();` no final do `app.js` havia sido perdida.

Correção: chamada `exibirSaldo();` adicionada no final do arquivo.

Lição: um código pode estar funcionando (guardando certo) mas não fazer o que esperamos (mostrar). Por isso testamos sempre — é assim que se depura em projetos reais.

MICROETAPA 4 — O EXTRATO (ARRAY E JSON)

Objetivo: cada transação aparece na tela como item do Extrato, e a lista inteira é salva na gaveta.

```
<section>
  <h2>Extrato</h2>
  <ul id="lista-transacoes"></ul>
</section>
```

```
var transacoes = JSON.parse(localStorage.getItem("transacoes")) || [];

function salvarTransacoes() {
  localStorage.setItem("transacoes", JSON.stringify(transacoes));
}

function exibirTransacoes() {
  var lista = document.getElementById("lista-transacoes");
  lista.textContent = "";

  for (var i = 0; i < transacoes.length; i++) {
    var item = document.createElement("li");
    var transacao = transacoes[i];
    item.textContent = transacao.descricao + " — R$ " + transacao.valor.toFixed(2).repl
    lista.appendChild(item);
  }
}
```

Conceitos novos:

- **Array (lista) `[ ]`:** guarda vários valores em fila.
- **Objeto `{ chave: valor }`:** agrupa informações (descrição + valor).
- **`push(...)`:** adiciona item no final da lista.
- **Laço `for`:** `for (var i = 0; i < transacoes.length; i++)` percorre a lista; `transacoes[i]` é o item da vez; `length` = tamanho.
- **`createElement("li")`:** cria um item (ainda fora da tela).
- **`lista.appendChild(item)`:** anexa o item criado dentro da lista.

- **JSON:** a gaveta só guarda texto. `JSON.stringify` vira texto (salvar); `JSON.parse` volta para lista (ler).
- **`lista.textContent = ""`:** limpa antes de redesenhar (senão a lista dobrava).

🎉 Fim da Fase 2: o app soma, subtrai, mostra o histórico e não perde nada.

BUG CORRIGIDO — transação sem valor entrava no extrato

Sintoma: clicar em "Adicionar" com valor vazio criava a transação mesmo assim.

Diagnóstico: `Number("")` vira `0`, e não havia checagem antes de salvar. Campos de formulário sugerem, mas não bloqueiam — a proteção de verdade é o código.

Correção (validação), nos dois formulários:

```
if (valor <= 0) {
  alert("Digite um valor maior que zero.");
  return;
}
```

Conceitos novos: `if (condição)` = o "se" da programação; `alert("...")` = janelinha de aviso; `return;` = sai da função na hora, ignorando o resto (o "ponto de interdição").

Lição: todo dado que entra precisa ser **validado** antes de ser usado — base de segurança em qualquer app.

FASE 3 — CATEGORIAS E ORGANIZAÇÃO

Microetapa 1 — O seletor de categorias

Objetivo: adicionar um menu de escolha (dropdown) com categorias no formulário de despesas, para futuramente organizar e resumir os gastos.

Por que: sem categoria, o extrato é só uma lista solta. Com ela, poderemos responder "quanto gastei em alimentação?" — o coração de um controle financeiro.

```
<label for="categoria-despesa">Categoria</label>
<select id="categoria-despesa">
  <option>Moradia</option>
  <option>Alimentação</option>
  <option>Transporte</option>
  <option>Lazer</option>
  <option>Saúde</option>
  <option>Outros</option>
</select>
```

```
select {
  width: 100%;
  padding: 8px;
  border: 1px solid #374151;
  border-radius: 4px;
  background-color: #111827;
  color: #f9fafb;
}
```

Conceitos novos: `<select>` = caixa de menu (dropdown), com `<label>` e `id` como os outros campos; `<option>` = cada escolha possível (o primeiro já vem selecionado). O CSS repetiu as regras do `input` para que os campos fiquem com a mesma aparência.

Importante: nesta microetapa a categoria aparece na tela, mas **ainda não é salva** — isso é a próxima etapa (o JS precisa ler o valor do `<select>` e guardá-lo).

Microetapa 2 — Salvar a categoria em cada transação

Objetivo: o JS passar a ler o valor escolhido no menu e guardá-lo junto com a despesa.

```
var categoria = document.getElementById("categoria-despesa").value;
transacoes.push({ descricao: descricao, valor: valor, categoria: categoria });
```

- `document.getElementById("categoria-despesa").value` → lemos o valor escolhido no `<select>`. Todo campo tem `.value`: no `<input>` é o texto digitado; no `<select>` é a `<option>` selecionada.
- `categoria: categoria` → adicionamos um **novo campo** ao objeto da transação (`{ descricao, valor }` agora também tem `categoria`).
- **Não foi preciso mudar como salvamos:** como o objeto ganhou o campo novo, a lista inteira salva com JSON já o inclui automaticamente.

Microetapa 3 — Mostrar a categoria no extrato

Objetivo: mostrar a categoria só quando ela existir (receitas não têm categoria).

```
if (transacao.categoria) {
  item.textContent = transacao.descricao + " [" + transacao.categoria + "] — R$ " + tra
} else {
  item.textContent = transacao.descricao + " — R$ " + transacao.valor.toFixed(2).replac
}
```

- **`if` / `else`** → "se ... senão ...": um caminho quando a condição é verdadeira, outro quando é falsa.
- `if (transacao.categoria)` → além de `true`/`false`, o `if` aceita qualquer valor "verdadeiro"; se não existir, é `undefined` (falso) → cai no `else`.
- **Lição:** o mesmo tipo de dado pode ter **formatos diferentes**; o código precisa se adaptar.

🎉 Fim da Fase 3: as despesas agora estão organizadas por categoria.

FASE 4 — RESUMO E TOTAL POR CATEGORIA

Microetapa 1 — Tipo da transação e cores no extrato (verde/vermelho)

Objetivo: pintar de **verde** o que entra (receita) e de **vermelho** o que sai (despesa). Para isso, cada transação precisa "saber" o que ela é.

Adicionamos um campo `tipo` ao objeto da transação:

```
transacoes.push({ descricao: descricao, valor: valor, tipo: "receita" });
transacoes.push({ descricao: descricao, valor: valor, categoria: categoria, tipo: "desp
```

```
function ehDespesa(transacao) {
  return transacao.tipo === "despesa" || (!transacao.tipo && transacao.categoria);
}
```

```
var despesa = ehDespesa(transacao);
var sinal = despesa ? "- " : "+ ";
item.textContent = transacao.descricao + " — " + sinal + formatarDinheiro(transacao.val
item.className = despesa ? "despesa" : "receita";
```

```
.receita {
  color: #22c55e;
  font-weight: bold;
}

.despesa {
  color: #ef4444;
  font-weight: bold;
}
```

Conceitos novos:

- **Campo `tipo`:** guarda a "natureza" da transação — o dado que faltava para colorir e resumir.
- **`===`:** compara valor e tipo (diferente de `=`, que *atribui*).
- **Operador ternário `condição ? a : b`:** versão curta do `if/else`.
- **`item.className`:** troca a classe CSS do elemento criado, ligando-o a `.receita` / `.despesa`.
- **Função auxiliar (DRY):** pequena função reaproveitada em vários pontos, para não repetir código.
- **Retrocompatibilidade:** `ehDespesa` também reconhece despesas antigas (sem o campo `tipo`), usando a categoria como pista.

Microetapa 2 — A seção Resumo (total de receitas e despesas)

Objetivo: mostrar separadamente quanto entrou e quanto saiu, além do saldo final.

```
<section>
  <h2>Resumo</h2>
  <p>Receitas: <span id="total-receitas" class="receita"></span></p>
  <p>Despesas: <span id="total-despesas" class="despesa"></span></p>
</section>
```

```
function exibirResumo() {
  var totalReceitas = 0;
  var totalDespesas = 0;

  for (var i = 0; i < transacoes.length; i++) {
    if (ehDespesa(transacoes[i])) {
      totalDespesas = totalDespesas + transacoes[i].valor;
    } else {
      totalReceitas = totalReceitas + transacoes[i].valor;
    }
  }

  document.getElementById("total-receitas").textContent = formatarDinheiro(totalReceita
  document.getElementById("total-despesas").textContent = formatarDinheiro(totalDespesa
}
```

Conceitos novos:

- **Acumulador:** variável que começa em `0` e recebe soma a cada volta do laço (`total = total + valor`). É assim que se calcula total em qualquer sistema.
- **Cor com significado:** verde/vermelho comunicam rápido (entrada x saída).

Microetapa 3 — Gastos por categoria

Objetivo: responder "quanto gastei em cada categoria?".

```
<section>
  <h2>Gastos por categoria</h2>
  <ul id="lista-categorias"></ul>
</section>
```

```
function exibirCategorias() {
  var lista = document.getElementById("lista-categorias");
  lista.textContent = "";

  var totais = {};

  for (var i = 0; i < transacoes.length; i++) {
    var transacao = transacoes[i];

    if (ehDespesa(transacao)) {
      var categoria = transacao.categoria;

      if (totais[categoria]) {
        totais[categoria] = totais[categoria] + transacao.valor;
      } else {
        totais[categoria] = transacao.valor;
      }
    }
  }

  for (var categoria in totais) {
    var item = document.createElement("li");
    item.textContent = categoria + " — " + formatarDinheiro(totais[categoria]);
    lista.appendChild(item);
```

```
  }
}
```

Conceitos novos:

- **Objeto como dicionário:** `totais[categoria]` usa o *valor da variável* como chave.
- **Criar a chave na primeira vez:** se ainda não existe, guarda o primeiro valor; depois, soma (padrão "inicializa e acumula").
- **Laço `for...in`:** percorre as **chaves** de um objeto (aqui, cada categoria).

🎉 Fim da Fase 4: saldo, resumo com cores e gastos por categoria.

FASE 5 — APERFEIÇOAMENTOS E DIFERENCIAIS

Microetapa 1 — Importação de extrato em CSV

Objetivo: em vez de digitar transação por transação, o usuário escolhe um arquivo `.csv` e o app lança todas de uma vez — o "diferencial" do projeto.

Formato adotado (uma transação por linha), colunas separadas por vírgula e decimal com ponto:

```
descricao,valor,tipo,categoria
Salario,3000,receita,
Mercado,450.75,despesa,Alimentação
Aluguel,1200,despesa,Moradia
```

```
<section>
  <h2>Importar extrato (CSV)</h2>
  <p class="dica">Formato: descricao,valor,tipo,categoria (uma transação por linha).</p
  <input type="file" id="arquivo-csv" accept=".csv">
</section>
```

Primeiro, uma **refatoração** (reduzir repetição): extraímos a lógica comum para funções reutilizáveis.

```
function registrarTransacao(descricao, valor, tipo, categoria) {
  if (tipo === "despesa") {
    saldo = saldo - valor;
  } else {
    saldo = saldo + valor;
  }

  var transacao = { descricao: descricao, valor: valor, tipo: tipo };
```

```
  if (categoria) {
    transacao.categoria = categoria;
  }

  transacoes.push(transacao);
}

function atualizarTela() {
  exibirSaldo();
  salvarTransacoes();
  exibirTransacoes();
  exibirResumo();
  exibirCategorias();
}
```

```
document.getElementById("arquivo-csv").addEventListener("change", function (evento) {
  var arquivo = evento.target.files[0];

  if (!arquivo) {
    return;
  }

  var leitor = new FileReader();

  leitor.onload = function () {
    importarCSV(leitor.result);
  };

  leitor.readAsText(arquivo, "UTF-8");
});
```

```
function importarCSV(texto) {
  var linhas = texto.split(/\r?\n/);
  var importadas = 0;

  for (var i = 0; i < linhas.length; i++) {
    var linha = linhas[i].trim();

    if (linha === "") {
      continue;
    }
```

```
    var partes = linha.split(",");
    var descricao = partes[0].trim();
    var valor = Number(partes[1]);
    var tipo = (partes[2] || "").trim().toLowerCase();
    var categoria = (partes[3] || "").trim();

    if (descricao === "" || isNaN(valor) || valor <= 0) {
      continue;
    }

    if (tipo !== "receita" && tipo !== "despesa") {
      continue;
    }

    registrarTransacao(descricao, valor, tipo, categoria);
    importadas = importadas + 1;
  }

  atualizarTela();
  alert(importadas + " transação(ões) importada(s).");
}
```

Conceitos novos:

- **`<input type="file">`:** abre a janela para escolher um arquivo; `accept=".csv"` filtra o tipo; o arquivo fica em `evento.target.files[0]`.
- **`FileReader`:** lê arquivos no navegador. `readAsText(arquivo, "UTF-8")` lê como texto; ao terminar, `onload` roda e o conteúdo está em `leitor.result` (assíncrono).
- **`split(/\r?\n/)`:** quebra em linhas; `/\r?\n/` é uma **expressão regular** que aceita quebras do Windows e do Linux.
- **`.trim()`:** remove espaços das pontas de um texto.
- **`isNaN(valor)`:** "não é número"; usamos para descartar linhas inválidas (e o cabeçalho).
- **`continue;`:** pula para a próxima volta do laço na hora (diferente do `return`, que sai da função).
- **`(partes[2] || "")`:** evita erro quando a célula não existe.
- **Refatoração / DRY:** `registrarTransacao` e `atualizarTela` centralizam a lógica; digitar e importar usam o mesmo caminho.

Teste: há um arquivo `exemplo-extrato.csv` na pasta do projeto — escolha-o na tela de importação.

Lição: todo dado que vem de fora precisa ser **validado**; linhas erradas são ignoradas sem quebrar o app.

Microetapa 2 — Open Finance simulado

Objetivo: simular a conexão com um banco — escolher a instituição, autorizar o compartilhamento e "sincronizar" as movimentações.

Por que "simulado": o Open Finance de verdade exige registro no Banco Central, certificados e regras de LGPD. Aqui representamos a **ideia** com bancos e movimentações fictícios, guardados no próprio código. O Open Finance real é o sistema do Banco Central que permite compartilhar seus dados entre instituições **com o seu consentimento**.

```
<section>
  <h2>Open Finance (simulado)</h2>
  <p class="dica">Conexão fictícia com um banco. Nenhum dado real é usado.</p>
  <label for="banco-openfinance">Instituição</label>
  <select id="banco-openfinance">
    <option>Banco Aurora</option>
    <option>Banco Laranja</option>
    <option>Fintech Pingo</option>
  </select>
  <label class="consentimento">
    <input type="checkbox" id="consentimento-openfinance">
    Autorizo o compartilhamento dos meus dados (simulado)
  </label>
  <button type="button" id="btn-openfinance">Conectar e importar</button>
  <p id="status-openfinance"></p>
</section>
```

"Banco de dados" da simulação: um objeto cujos valores são **listas** de movimentações.

```
var contasOpenFinance = {
  "Banco Aurora": [
    { descricao: "Pix recebido", valor: 250, tipo: "receita" },
    { descricao: "Supermercado", valor: 320.5, tipo: "despesa", categoria: "Alimentação
    { descricao: "Posto de gasolina", valor: 180, tipo: "despesa", categoria: "Transpor
  ],
  "Banco Laranja": [ /* ... */ ],
```

```
  "Fintech Pingo": [ /* ... */ ]
};
```

```
document.getElementById("btn-openfinance").addEventListener("click", function () {
  var banco = document.getElementById("banco-openfinance").value;
  var consentimento = document.getElementById("consentimento-openfinance").checked;
  var status = document.getElementById("status-openfinance");
  var botao = document.getElementById("btn-openfinance");

  if (!consentimento) {
    alert("Marque a autorização para compartilhar os dados (simulado).");
    return;
  }

  botao.disabled = true;
  status.textContent = "Conectando ao " + banco + "...";

  setTimeout(function () {
    var movimentacoes = contasOpenFinance[banco];

    for (var i = 0; i < movimentacoes.length; i++) {
      var movimentacao = movimentacoes[i];
      registrarTransacao(movimentacao.descricao, movimentacao.valor, movimentacao.tipo,
    }

    atualizarTela();
    status.textContent = movimentacoes.length + " transação(ões) sincronizada(s) do " +
    botao.disabled = false;
  }, 1500);
});
```

Conceitos novos:

- **Simulação (mock):** imitar algo real (um banco) sem depender dele — muito usado para testar e prototipar.
- **Consentimento:** o usuário precisa autorizar; no Open Finance real isso é obrigatório (LGPD).
- **`.checked`:** estado de um `<input type="checkbox">` (`true`/`false`).
- **`.disabled`:** desliga o botão enquanto conecta (evita clique duplo).

- **`setTimeout(função, 1500)`:** roda a função depois de 1,5 s (**assíncrono**) — simula o tempo de conexão.
- **Objeto com listas dentro:** `contasOpenFinance[banco]` devolve a lista de movimentações daquele banco.
- **`button type="button"`:** botão que não envia formulário.

A grande sacada: a sincronização **reaproveita** `registrarTransacao` e `atualizarTela` (Microetapa 1). Como a lógica já estava centralizada, adicionar um banco novo não exigiu mexer no saldo, no extrato nem no resumo — **o valor da refatoração aparecendo**.

Lição: em sistemas reais, integrar com terceiros envolve consentimento, tratamento de erros e tempo de espera.

FASE 6 — ENTRADA DO SITE: LANDING + CADASTRO/LOGIN

Até agora, ao abrir o `index.html`, você caía direto no app. Nesta fase o site ganha uma **porta de entrada**: uma página de apresentação (landing), telas de **cadastro** e **login**, e o app só abre para quem está logado.

Microetapa 1 — Separar o app em app.html

A página que já existia (o app) vira `app.html`, e o nome `index.html` passa a ser a **landing**. Assim separamos "entrada do site" de "aplicativo":

Arquivo	Papel
index.html	landing (página de apresentação)
cadastro.html / login.html	criar conta / entrar
app.html	o aplicativo (saldo, extrato, resumo...)

Conceito novo — site multi-página: o projeto deixou de ser uma página só e passou a ser várias, ligadas por links (`<a href="...">`) e pelo JavaScript (`window.location.href = "app.html"`).

Importante: o `localStorage` é **compartilhado entre as páginas** da mesma pasta/origem — por isso a sessão atravessa da tela de login para o app.

Microetapa 2 — A biblioteca auth.js

Um arquivo só com as funções de autenticação, reaproveitado por todas as páginas — uma **caixa de ferramentas compartilhada**. Incluído com `<script src="auth.js"></script>` antes do script de cada página.

- `carregarUsuarios()` / `salvarUsuarios(lista)` — leem e gravam a lista de contas (chave `cf_usuarios`).
- `hashSenha(senha)` — transforma a senha em um **código** (hash).
- `existeEmail(email)` — evita cadastrar o mesmo e-mail duas vezes.
- `sessaoEmail()` / `definirSessao(email)` / `sair()` — guardam/limpam quem está logado (chave `cf_sessao`).
- `usuarioAtual()` — devolve os dados do usuário logado.
- `exigirLogin()` — se não há sessão, manda o navegador para o login.

Atenção: o nosso hash é **simples e didático**. Em um app real a senha é protegida **no servidor** (algoritmos como bcrypt), com HTTPS e LGPD.

Microetapa 3 — Cadastro (cadastro.html + cadastro.js)

Formulário com **nome, e-mail e senha**. Ao enviar: `preventDefault()`; valida campos vazios; `existeEmail` bloqueia repetido; monta o usuário com `senha: hashSenha(senha)`; `salvarUsuarios` grava; mostra um `alert` de sucesso e vai para o `login.html`.

Microetapa 4 — Login (login.html + login.js)

Lê e-mail e senha, procura o usuário e compara `hashSenha(senha)` com a senha guardada. Se bater, `definirSessao(email)` e vai para o `app.html`; se não, avisa "e-mail ou senha inválidos". **Lembrete:** não revelamos *qual* dos dois errou — isso não dá pistas para adivinhar senhas.

Microetapa 5 — Proteger o app + dados por usuário

A **primeira linha** do `app.js` é `exigirLogin();` — quem não tem sessão nunca vê o app. E as chaves do `localStorage` ganham o e-mail do usuário:

```
var usuario = usuarioAtual();
var prefixo = "cf_" + usuario.email + "_";
var saldo = Number(localStorage.getItem(prefixo + "saldo")) || 0;
```

Assim cada conta tem seus próprios dados (`cf_maria@..._saldo`, `cf_joao@..._saldo`...). O cabeçalho mostra "Olá, <nome>!" e um botão **Sair** que chama `sair()`.

Microetapa 6 — A landing (index.html + entrar.css)

Página de apresentação com cabeçalho (Entrar / Criar conta), um **destaque (hero)**, uma **faixa de categorias** que rola, **cartões de recursos** e um convite final.

Estilo: inspirado no site **valuationia.com.br** (visual editorial: título grande, cantos retos, grade sutil no fundo, cartões que "sobem" no hover), **mas com as cores do projeto** (fundo #111827, cartões #1f2937, destaque laranja #f97316, texto #f9fafb).

Conceitos novos de CSS: variáveis em `:root`; grade de fundo com `linear-gradient`; `mask-image` (recorte com degradê); `@keyframes` + `animation` (entrada e faixa rolante); `-webkit-text-stroke` (texto vazado); fontes do Google (Plus Jakarta Sans e JetBrains Mono).

Teste: abra o `index.html` → Criar conta → cadastre → faça login → o app abre com "Olá, ...". Recarregue (F5): continua logado. Clique em **Sair**: volta para a landing e o app deixa de abrir.

Lição: adicionar login mudou a **arquitetura** do site (de 1 para várias páginas) e ensinou dois temas de ouro: **reutilizar código** (auth.js) e **organizar dados** (prefixo por usuário).

GITHUB — SUBINDO O PROJETO PARA A NUVEM

Git = a "máquina do tempo" (registra versões, dá para voltar no tempo). **GitHub** = serviço grátis que guarda o repositório na nuvem (backup + compartilhamento).

Passos feitos:

- `git init` → transformou a pasta em repositório.
- `git config user.name` / `user.email` → identidade para os commits.
- `git add .` → "empacotou" os arquivos (staging).
- `git commit -m "..."` → tirou a "fotografia" (versão salva).
- `git branch -m main` → branch no padrão do GitHub.
- `git remote add origin <url>` → conectou com o repositório na nuvem.
- `git push -u origin main` → **enviou tudo.**

Guardar uma etapa nova: `git add .` → `git commit -m "mensagem"` → `git push`.

Conceitos: commit = versão registrada; branch = linha de desenvolvimento (main); remote = endereço da nuvem; push = enviar; repositório = pasta + máquina do tempo.

Link do projeto: https://github.com/taylorhdrescher/Controle-Financeiro (botão "commits" mostra a linha do tempo).

GLOSSÁRIO — Conceitos aprendidos

Conceito	O que é	Onde usamos
Extensão de arquivo	Final do nome que identifica o tipo	.html, .css, .js
Tag em par	Elemento HTML que abre e fecha	...
Aninhamento	Caixas dentro de caixas	<h1> dentro de <header>
Variável	Caixa na memória que guarda valor	var saldo

Função	Bloco que executa quando chamado	function exibirSaldo()
Id	Endereço único de um elemento	id="saldo"
Evento	Sinal do navegador (clique etc.)	addEventListener("submit")
preventDefault	Impede o comportamento padrão	parar recarga do form
.value	Lê o que foi digitado no campo	getElementById("valor").value
Number()	Converte texto em número	Number("250") → 250
toFixed(2)	Força 2 casas decimais	R$ 0,00
Media query	Regra CSS condicionada ao tamanho da tela	responsividade
localStorage	Gaveta de memória do navegador	salvar entre F5
Persistência	Guardar dados além do programa rodando	saldo não some
Array	Lista que guarda vários valores	var transacoes = [...]
Objeto	Agrupa informações sobre a mesma coisa	{ descricao, valor }
push()	Adiciona item ao final da lista	transacoes.push(...)
length	Tamanho de uma lista	transacoes.length
Laço for	Repete um bloco várias vezes	percorrer a lista
createElement	Cria um elemento no JS	criar
appendChild	Anexa o elemento criado na página	colocar na lista

JSON	Formato texto que representa dados	salvar a lista na gaveta
Validação	Checar o dado antes de usar	if (valor <= 0)
if	O "se" da programação	condição
alert()	Janelinha de aviso do navegador	"Digite um valor maior..."
return	Sai da função na hora	interromper envio inválido
Git / commit	Máquina do tempo / versão registrada	backup do projeto
GitHub / push	Nuvem / enviar para a nuvem	repositório online
select / option	Menu de escolha (dropdown)	categoria da despesa
Campo tipo	Natureza da transação (receita/despesa)	colorir e resumir
===	Compara valor e tipo	tipo === "despesa"
Operador ternário	Versão curta do if/else	despesa ? "- " : "+ "
className	Define a classe CSS via JS	.receita / .despesa
Acumulador	Variável que soma no laço	total de receitas
for...in	Percorre as chaves de um objeto	cada categoria
Objeto como dicionário	Usa o valor da variável como chave	totais[categoria]
Função auxiliar (DRY)	Reaproveita lógica, evita repetição	formatarDinheiro / ehDespesa
if / else	"se ... senão ..."	item com ou sem categoria
input file	Escolher arquivo do computador	importar CSV

FileReader	Lê arquivos no navegador (assíncrono)	readAsText + onload
split()	Quebra um texto em partes	quebrar CSV em linhas/células
trim()	Remove espaços das pontas	limpar células
isNaN()	Diz se o valor "não é número"	descartar linha inválida
continue	Pula para a próxima volta do laço	ignorar linha errada
Expressão regular	Padrão de busca em texto	/\r?\n/ (quebras de linha)
Refatoração	Reorganizar sem mudar o resultado	registrarTransacao / atualizarTela
Simulação (mock)	Imitar algo real sem depender dele	bancos fictícios
Consentimento	Autorizar o uso dos dados (LGPD)	checkbox do Open Finance
.checked	Estado de um checkbox (true/false)	autorização marcada
.disabled	Desliga/liga um botão	botão enquanto conecta
setTimeout	Roda uma função depois de um tempo	simular "conectando..."
type="button"	Botão que não envia formulário	conectar ao banco
Site multi-página	Várias páginas .html ligadas por links	index → login → app
window.location.href	Manda o navegador para outra página	ir para app.html
Biblioteca (script compartilhado)	Arquivo só de funções usado em várias páginas	auth.js

Hash de senha	Código gerado a partir da senha	hashSenha()
Sessão	Marca de quem está logado	chave cf_sessao
exigirLogin	Protege a página: expulsa quem não tem sessão	1ª linha do app.js
Prefixo por usuário	Chave do localStorage com o e-mail	cf_<email>_saldo
Variáveis CSS (:root)	Valores reutilizáveis no CSS	--accent: #f97316
@keyframes / animation	Animações em CSS	entrada e faixa rolante
mask-image	Recorte suave com degradê	fade do fundo/faixa
-webkit-text-stroke	Texto só com contorno (vazado)	destaque do hero

PRÓXIMOS PASSOS

- Fase 3 — Categorias e organização ✅ concluída.
  - [x] Microetapa 1: seletor de categorias no formulário.
  - [x] Microetapa 2: salvar a categoria em cada transação.
  - [x] Microetapa 3: mostrar a categoria no extrato.
- Fase 4 — Resumo e total por categoria ✅ concluída.
  - [x] Microetapa 1: tipo da transação e cores no extrato.
  - [x] Microetapa 2: seção Resumo (receitas e despesas).
  - [x] Microetapa 3: gastos por categoria.
- Fase 5 — Aperfeiçoamentos e diferenciais ✅ concluída.
  - [x] Microetapa 1: importação de extrato em CSV.
  - [x] Microetapa 2: Open Finance simulado.
- Fase 6 — Entrada do site (landing + cadastro/login) ✅ concluída.
  - [x] Microetapa 1: separar o app em app.html.
  - [x] Microetapa 2: biblioteca auth.js.
  - [x] Microetapa 3: cadastro.
  - [x] Microetapa 4: login.

    - [x] Microetapa 5: proteger o app + dados por usuário.
    - [x] Microetapa 6: landing no estilo valuationia (cores do projeto).
- Próximas ideias: aplicar o visual da landing também ao app, metas de gasto por categoria, exportar relatório.